TaskTrack Database Design

Entity Relationship Diagram, table inventory, primary keys, foreign keys, and application-level references

Purpose and scope

This report summarizes the database structure used by the TaskTrack system. It is based on the project schema snapshot and describes only relationships explicitly declared as foreign keys as enforced database relationships. It is not a generic student/course sample.

At a glance

17 tables in schema snapshot	**5** declared foreign-key constraints	**0** declared links from most task/history tables	**2** unique keys on users: email and employee_id

Declared relationships

Parent table	Child table	Foreign-key column	Delete behavior
users	human_resource	user_id -> users.id	ON DELETE CASCADE
attendance	human_resource	attendance_id -> attendance.attendance_id	ON DELETE CASCADE
employee	payroll	employee_id -> employee.employee_id	ON DELETE CASCADE
production	logistics	production_id -> production.production_id	ON DELETE CASCADE
users	supervisor	user_id -> users.id	ON DELETE CASCADE

Important: A column named like another table ID is not automatically a foreign key. For example, users.employee_id is unique when populated, but the schema does not declare it as a foreign key to employee.employee_id. Similar links are documented separately in the table catalog.

Source: config/install_schema.sql and listed schema migrations

Entity Relationship Diagram

All schema tables are shown. Solid arrows represent declared foreign keys only; un-enforced references are labeled in the entity cards.

Solid arrows = foreign keys enforced by MySQL

*Application reference only; not a declared database foreign key

Source: config/install_schema.sql and listed schema migrations

Table and key inventory

PK = primary key; FK = declared foreign key; UK = unique key. Tables without an FK listed have no declared foreign-key constraint in the schema snapshot.

Table	Primary key	Foreign key(s) / notes
applicant	applicant_id	None
attendance	attendance_id	None (user_id is indexed; user_id and employee_id are not FKs)
delivery	delivery_id	None (production_id, driver_id and reviewer/assigner IDs are not FKs)
delivery_activity	activity_id	None (delivery and actor/driver IDs are not FKs)
employee	employee_id	None
employee_leave_requests	leave_id	None (employee_id, recorded_by, cancelled_by are not FKs)
human_resource	hr_id	user_id -> users.id; attendance_id -> attendance.attendance_id
logistics	logistics_id	production_id -> production.production_id
office	office_id	None (employee, supervisor, reviewer IDs are not FKs)
payroll	payroll_id	employee_id -> employee.employee_id (attendance_id is not an FK)
production	production_id	None
production_activity	activity_id	None (production and actor/employee IDs are not FKs)
production_assignments	production_id + work_type (composite)	None (employee_id, assigned_by, reviewer IDs are not FKs)
supervisor	supervisor_id	user_id -> users.id
system_audit_log	audit_id	None (actor_user_id is not an FK)
task_notifications	notification_id	None (user_id and actor_user_id are not FKs)
users	id	None; UK: email, employee_id

Interpretation: The system uses additional application-level references for task assignment, delivery, audit, notification, attendance, leave, and history records. They may be joined by application code, but MySQL will not enforce referential integrity for them. In particular, production_assignments.production_id and production_activity.production_id are stored as VARCHAR while production.production_id is INT.

Source: config/install_schema.sql and listed schema migrations